1. **What is the Event Loop in Node.js, and how does it work?**

⇨ The **Event Loop** in Node.js is a mechanism that allows **non-blocking, asynchronous operations** despite JavaScript being **single-threaded**. It continuously cycles through **different phases** (Timers, Pending Callbacks, Poll, Check, Close) to execute queued tasks like I/O operations, timers, and promises.

2. **How is Node.js single-threaded but still handles concurrent operations?**

⇨ Node.js is **single-threaded** but handles **concurrent operations** using the **Event Loop** and **asynchronous, non-blocking I/O**.

3. **What are phases in the Event Loop, and how do they affect execution order?**

⇨ The Event Loop in Node.js has six phases: **Timers**, which executes `setTimeout()` and `setInterval()` callbacks; **Pending Callbacks**, which handles deferred I/O callbacks; **Idle/Prepare**, used internally; **Poll**, which processes I/O operations and waits for new events; **Check**, where `setImmediate()` callbacks run; and **Close Callbacks**, which handles cleanup tasks like socket closures. The loop also prioritizes **Microtasks** (`process.nextTick()`, `Promise.then()`) before moving to the next phase, ensuring efficient execution of asynchronous tasks.

4. **How does the setTimeout() function work in the Event Loop?**

⇨ `setTimeout()` schedules a task to run after a delay but doesn't guarantee exact timing. It is handled in the **Timers phase** of the Event Loop. If the Call Stack is busy, execution may be delayed until the next available cycle.

5. **What is the difference between process.nextTick() and setImmediate()**

Execution Timing: `process.nextTick()` runs **immediately after the current operation**, before the Event Loop continues, while `setImmediate()` runs in the **Check phase** of the next Event Loop cycle.

Priority: `process.nextTick()` has **higher priority** and executes before `setImmediate()`.

Use Case: Use `process.nextTick()` for **urgent microtasks**, and `setImmediate()` for **I/O-related callbacks** that should run after the Poll phase.

6. **What are callbacks in Node.js? How do they help in asynchronous programming?**

Callbacks are functions passed as arguments to other functions and executed later. They

enable **asynchronous programming** by allowing tasks like file reading, API calls, or database queries to run **without blocking** the main thread.

7. What is callback hell, and how can you avoid it?
Callback hell occurs when multiple nested callbacks make code **difficult to read and maintain**. It can be avoided by using **Promises, async/await, or breaking code into smaller functions** to improve readability.

8. What are Promises in Node.js, and how do they work?
A Promise represents a **future value** of an asynchronous operation and can be in **pending, fulfilled, or rejected** states. Instead of deeply nested callbacks, Promises allow chaining (`.then()`, `.catch()`) for better readability and error handling.

9. How is async/await different from Promises, and when should you use it?
Async/await is a **simpler and cleaner** way to handle Promises, making asynchronous code look **synchronous**. It eliminates `.then()` chains and improves readability. Use async/await when you need **sequential execution** or better error handling with `try-catch`.

10. What happens if you don't use try-catch inside an async/await function?
If an error occurs inside an async function without `try-catch`, the **Promise rejects** and may cause an **unhandled rejection**, potentially crashing the application. To prevent this, always handle errors properly using `try-catch` or `.catch()`.

11. What is the difference between CommonJS and ES Modules in Node.js?
CommonJS (`require`) is the default module system in Node.js, while ES Modules (`import/export`) follow modern JavaScript standards. CommonJS is synchronous and widely used, while ES Modules support asynchronous loading and are preferred for modern applications.

12. How do you import/export modules in Node.js using both CommonJS and ES Modules?
In CommonJS, use `require()` to import and `module.exports` to export. In ES Modules, use `import` and `export` statements. The module system used depends on the `"type"` field in `package.json` or file extensions.

13. What is the purpose of the require.cache in CommonJS?
`require.cache` stores loaded modules to **prevent reloading** them on subsequent `require()` calls, improving performance by avoiding redundant file system operations.

14. What is the package.json "type" field, and how does it affect module usage?
The `"type"` field determines the module system:

- `"type": "commonjs"` (default) enables CommonJS (`require`).
- `"type": "module"` enables ES Modules (`import/export`).
  It affects how files are interpreted in a Node.js project.

15. What are some important global objects in Node.js (__dirname, __filename, process, Buffer)?

- `__dirname`: The absolute path of the current directory.
- `__filename`: The absolute path of the current file.
- `process`: Provides information and control over the Node.js process.
- `Buffer`: Handles binary data efficiently.

16. What is the difference between synchronous (fs.readFileSync) and asynchronous (fs.readFile) file operations in Node.js?
Synchronous (`fs.readFileSync`) **blocks** execution until the file is read, while asynchronous (`fs.readFile`) is **non-blocking**, allowing other tasks to run while waiting for the file operation to complete.

17. How do you read a file asynchronously in Node.js?
Use the `fs.readFile` method, which performs a **non-blocking** read operation and executes a callback when the data is available.

18. What is the difference between fs.readFile and fs.createReadStream?
`fs.readFile` reads the **entire file into memory** before processing, which can be inefficient for large files. `fs.createReadStream` reads the file **in chunks**, making it more memory-efficient and suitable for large data.

19. How do you write data to a file using the fs module in both sync and async ways?
Use `fs.writeFileSync` for synchronous writing (blocks execution) and `fs.writeFile` for asynchronous writing (non-blocking).

20. What are some common error-handling strategies when working with the file system in Node.js?
Use `try-catch` for synchronous operations, handle errors in callbacks for asynchronous functions, check file existence before operations, and use event listeners for stream errors.

21. What are Streams in Node.js, and why are they useful?
Streams handle **continuous data flow** efficiently, allowing large files to be processed **in chunks** rather than loading everything into memory, improving performance and scalability.

22. Explain the four types of streams in Node.js: Readable, Writable, Duplex, and Transform.

- **Readable Streams**: Only allow reading (e.g., file reading).
- **Writable Streams**: Only allow writing (e.g., file writing).
- **Duplex Streams**: Support both reading and writing (e.g., sockets).
- **Transform Streams**: Modify data while reading/writing (e.g., compression).

23. How do you create a Readable Stream in Node.js?
Use the `fs.createReadStream` method to create a stream that reads data **in chunks** instead of loading the entire file at once.

24. How does piping (.pipe()) work in Node.js Streams?
Piping connects a **Readable Stream** to a **Writable Stream**, transferring data automatically **without buffering**, improving efficiency.

25. How do Transform Streams work?
Transform Streams **modify** data while it is being read and written, such as compressing, encrypting, or converting data formats.

26. What are Buffers in Node.js, and why are they needed?
Buffers are used to handle **binary data** directly in memory, especially useful when working with **streams, file systems, and network operations**.

27. How does Node.js handle binary data using Buffers?
Node.js **allocates raw memory** outside the V8 heap using `Buffer`, allowing efficient handling of **binary data** without needing conversions.

28. What are the benefits of using Streams instead of loading a full file into memory?
Streams process data **in chunks**, reducing **memory usage**, improving **performance**, and allowing **faster** handling of large files or network data.

29. How does the highWaterMark property affect stream performance?
`highWaterMark` defines the **buffer size limit** for a stream. A **larger value** can improve throughput but use more memory, while a **smaller value** reduces memory usage but may increase latency.

30. What happens if a Writable Stream's write buffer fills up faster than it is being processed?
The stream enters a **backpressure state**, slowing down writes or pausing data flow until the buffer is cleared.

31. How do you create a basic HTTP server in Node.js without using Express?
Use the built-in `http` module to create a server that listens for client requests and sends responses.

32. What is the purpose of the http.createServer() method in Node.js?
It creates an HTTP server that listens for **incoming requests** and allows handling **routes, headers, and responses**.

33. How does req.url and req.method help in handling different routes in a basic HTTP server?
`req.url` helps identify the **requested endpoint**, and `req.method` determines whether the request is a **GET, POST, PUT, or DELETE**, allowing custom handling.

34. How can you handle query parameters and request body in a Node.js HTTP server?
Query parameters are extracted from `req.url`, while the request body is handled by **listening to data events** in streams and parsing it accordingly.

35. What is the difference between HTTP and WebSockets in Node.js?
HTTP is **stateless** and handles **request-response cycles**, while WebSockets provide a **persistent, full-duplex connection**, enabling **real-time communication**.

36. What are the key benefits of using Express.js over the built-in http module?
Express.js simplifies server creation by providing **routing, middleware support, request handling, and easy integration with databases**, making development faster and more efficient.

37. How do you define routes in Express.js?
Routes are defined using methods like `app.get()` and `app.post()`, which specify the URL path and request handler.

38. What is the difference between app.get() and app.post() in Express?
`app.get()` handles **retrieving data**, while `app.post()` is used to **send data** to the server, usually for creating resources.

39. How do you send a JSON response in an Express route handler?
Use `res.json()` to send a JSON-formatted response to the client.

40. How do you handle 404 (Not Found) errors in Express.js?
Define a **catch-all route** that returns a 404 response when no other route matches the request.

41. What is middleware in Express.js, and how does it work?
Middleware functions **process requests before reaching route handlers**, modifying requests, responses, or handling errors.

42. What are some common built-in middleware functions in Express.js?
Common middleware includes `express.json()` for parsing JSON, `express.urlencoded()` for form data, and `express.static()` for serving static files.

43. How do you create custom middleware in Express?
Custom middleware is a function that executes between request arrival and response sending, modifying request/response objects as needed.

44. What is the difference between application-level, router-level, and error-handling middleware?
Application-level middleware applies to **all routes**, router-level middleware applies to **specific routes**, and error-handling middleware deals with **errors** globally.

45. Name some commonly used third-party middleware in Express.js and their use cases.

- `cors` → Enables Cross-Origin Resource Sharing.
- `morgan` → Logs HTTP requests.
- `helmet` → Enhances security by setting HTTP headers.
- `body-parser` → Parses request bodies.

46. What are the key principles of REST APIs?
REST APIs follow **statelessness, resource-based URLs, HTTP methods, structured responses, and standard status codes**.

47. What are the different HTTP methods, and when should they be used?

- `GET` → Retrieve data.
- `POST` → Create new resources.
- `PUT` → Update or replace resources.
- `PATCH` → Partially update resources.
- `DELETE` → Remove resources.

48. What is idempotency in REST APIs, and which methods should be idempotent?
Idempotency means multiple identical requests result in the **same outcome**. `GET`, `PUT`, `DELETE`, and `PATCH` should be idempotent.

49. What is the difference between PUT and PATCH?
`PUT` **replaces** the entire resource, while `PATCH` **updates specific fields** of a resource.

50. How do you handle status codes correctly in a REST API?
Use appropriate HTTP status codes (`200` for success, `400` for bad request, `404` for not found, `500` for server errors) to indicate request outcomes.

51. What is JWT (JSON Web Token), and why is it used?
JWT is a compact, secure token used for **authentication and authorization**, ensuring safe data exchange between parties.

52. Explain the structure of a JWT token (Header, Payload, Signature).

- **Header** → Contains algorithm and token type.
- **Payload** → Stores user data (claims).
- **Signature** → Ensures integrity using a secret key.

53. How do you sign and verify JWTs in Node.js using jsonwebtoken?
Signing creates a token using a secret key, and verification checks its validity before granting access.

54. How do you securely store and transmit JWT tokens?
Store JWTs in **HTTP-only cookies** or **secure local storage**, and transmit them using **HTTPS** to prevent interception.

55. What are the common security risks when using JWTs?
Common risks include **token theft, exposure of sensitive data, insufficient expiration time, lack of encryption, and weak secret keys**

56. What is the difference between authentication and authorization?
Authentication verifies **who a user is**, while authorization determines **what they can access**.

57. How do you implement an authentication middleware in Express.js?
By checking **user credentials or JWT tokens** in requests and granting or denying access accordingly.

58. How can you protect API routes using JWT middleware?
Use a middleware function to **verify the JWT token** before allowing access to protected routes.

59. What is the purpose of refresh tokens, and how do they work?
Refresh tokens **extend user sessions** by allowing new access tokens to be issued without requiring re-login.

60. What are the advantages and disadvantages of session-based authentication vs. JWT authentication?

- **Session-based**: Secure, but requires **server storage** and scaling challenges.
- **JWT-based**: Stateless and scalable, but tokens can be **compromised if not securely handled**.

61. How do you connect Node.js to MongoDB using Mongoose?
By using Mongoose's `connect()` method to establish a database connection.

62. What is a Schema in Mongoose, and how does it differ from a Model?
A **Schema** defines the structure of documents, while a **Model** provides an interface to interact with MongoDB.

63. How do you define a MongoDB schema using Mongoose?
By specifying field types, validations, and default values in a Mongoose Schema.

64. What is the difference between find(), findOne(), and findById() in Mongoose?

- `find()` → Retrieves multiple documents.
- `findOne()` → Retrieves a single document.
- `findById()` → Finds a document by its **ObjectId**.

65. How do you perform an update operation in MongoDB using Mongoose?
Using methods like `updateOne()`, `updateMany()`, or `findByIdAndUpdate()`.

66. What are indexes in MongoDB, and how do they improve performance?
Indexes **speed up queries** by allowing faster data retrieval but require additional storage.

67. How do you handle schema validation in Mongoose?
Using built-in validation rules or **custom validation functions** within the Schema.

68. What is populate() in Mongoose, and how is it used for relationships?
`populate()` helps **fetch referenced documents** from another collection, enabling relational data retrieval.

69. How do you connect Node.js to PostgreSQL using pg (node-postgres)?
By using the `pg` library to establish a connection and execute queries.

70. What is a SQL transaction, and why is it important?
A transaction ensures **multiple queries execute as a single unit**, maintaining data integrity.

71. How do you perform a transaction in PostgreSQL using Node.js?
By using SQL commands like `BEGIN`, `COMMIT`, and `ROLLBACK` within a database connection.

72. What is the difference between INNER JOIN, LEFT JOIN, and RIGHT JOIN in SQL?

- **INNER JOIN** → Returns matching records in both tables.
- **LEFT JOIN** → Returns all records from the left table and matching ones from the right.
- **RIGHT JOIN** → Returns all records from the right table and matching ones from the left.

73. How do you prevent SQL injection in a Node.js application?
By using **parameterized queries** and **ORMs** to avoid directly injecting user input.

74. What is query optimization, and how can you improve SQL performance?
Query optimization improves performance using **indexes, proper joins, caching, and query tuning**.

75. What are indexes in PostgreSQL, and when should they be used?
Indexes **speed up searches** and should be used on columns frequently used in queries and filters.

76. What is clustering in Node.js, and how does it improve performance?
Clustering **creates multiple child processes** to utilize multiple CPU cores, improving performance by handling more requests concurrently

77. How does load balancing work in a Node.js application?
Load balancing distributes incoming requests **across multiple servers or processes** to improve performance and availability.

78. What is the difference between process-based and thread-based concurrency in Node.js?

- **Process-based**: Uses multiple processes, each with its own memory, ensuring isolation.
- **Thread-based**: Uses multiple threads within a single process, sharing memory.

79. How do you handle high traffic efficiently in a Node.js application?
By using **clustering, load balancing, caching, optimized queries, and asynchronous processing**.

80. What are the benefits of using Redis caching in a Node.js app?
Redis improves performance by **storing frequently accessed data in memory**, reducing database load and response times.

81. What is event loop blocking, and how can you avoid it?
Event loop blocking occurs when **long-running synchronous tasks** delay execution of other operations. Avoid it by using **asynchronous functions, worker threads, or background processing**.